Assignment 7

Chul Hee Kim

2021 10 31

Problem 1

a-)

```
CCdata <- getDataSet("Cement Composition.csv")
f1 <- lm(Y ~ ., data=CCdata)
summary(f1)
```

```
##
## Call:
## lm(formula = Y ~ ., data = CCdata)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.1750 -1.6709  0.2508  1.3783  3.9254
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  62.4054    70.0710   0.891   0.3991
## X1            1.5511     0.7448   2.083   0.0708 .
## X2            0.5102     0.7238   0.705   0.5009
## X3            0.1019     0.7547   0.135   0.8959
## X4           -0.1441     0.7091  -0.203   0.8441
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.446 on 8 degrees of freedom
## Multiple R-squared:  0.9824, Adjusted R-squared:  0.9736
## F-statistic: 111.5 on 4 and 8 DF,  p-value: 0.0000004756
```

```
vif(f1)
```

```
##        X1        X2        X3        X4
##  38.49621 254.42317  46.86839 282.51286
```

Y = 62.405 + 1.551X_1 + 0.510X_2 + 0.102X_3 - 0.144X_4

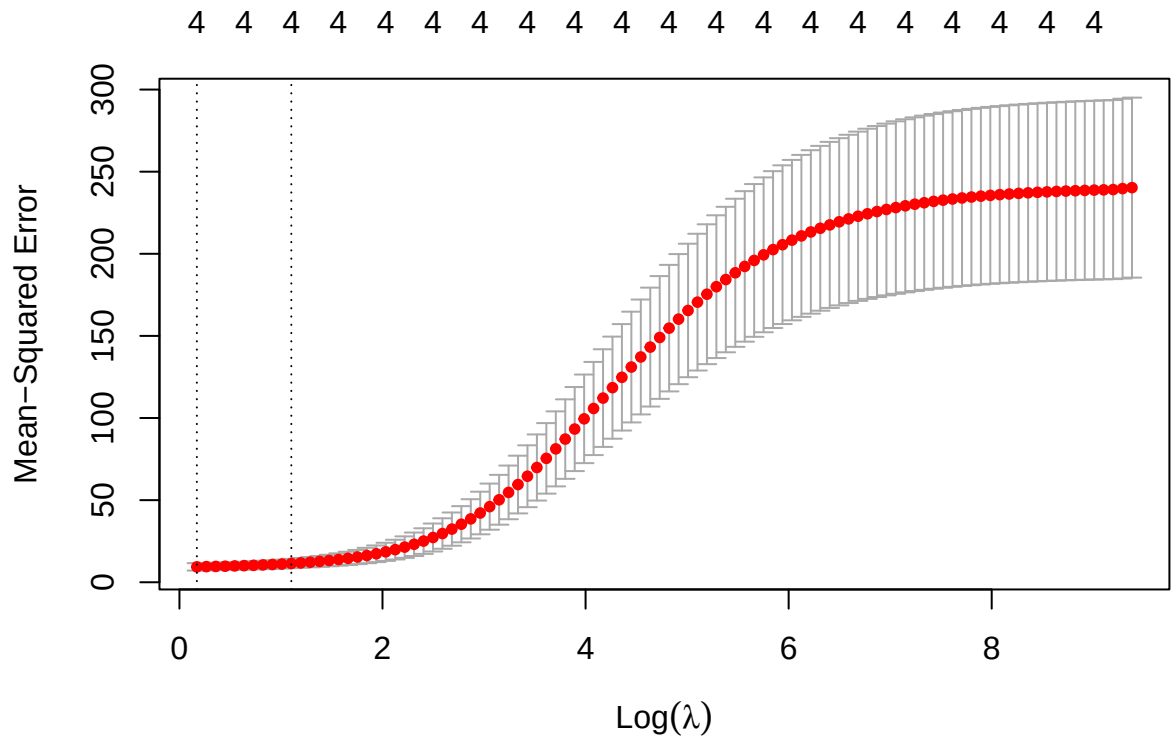
R^2 is very big, and the model is significant but none of the predictor variables are significant with α=0.05. VIFs indicate multicollinearity problem.

b-)

```
x <- model.matrix(Y~., CCdata)[,-c(1)]
y <- CCdata$Y
RidgeMod <- glmnet(x, y, alpha=0, nlambda=100,lambda.min.ratio=0.0001)
CvRidgeMod <- cv.glmnet(x, y, alpha=0, nlambda=100,lambda.min.ratio=0.0001)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
plot(CvRidgeMod)
```




```
best.lambda.ridge <- CvRidgeMod$lambda.min
pred.ridge <- coef(CvRidgeMod, s = "lambda.min")
pred.ridge
```

```
## 5 x 1 sparse Matrix of class "dgCMatrix"
##                      s1
## (Intercept) 86.4729570
## X1           1.1284667
## X2           0.2898568
## X3          -0.2558168
## X4          -0.3472310
```

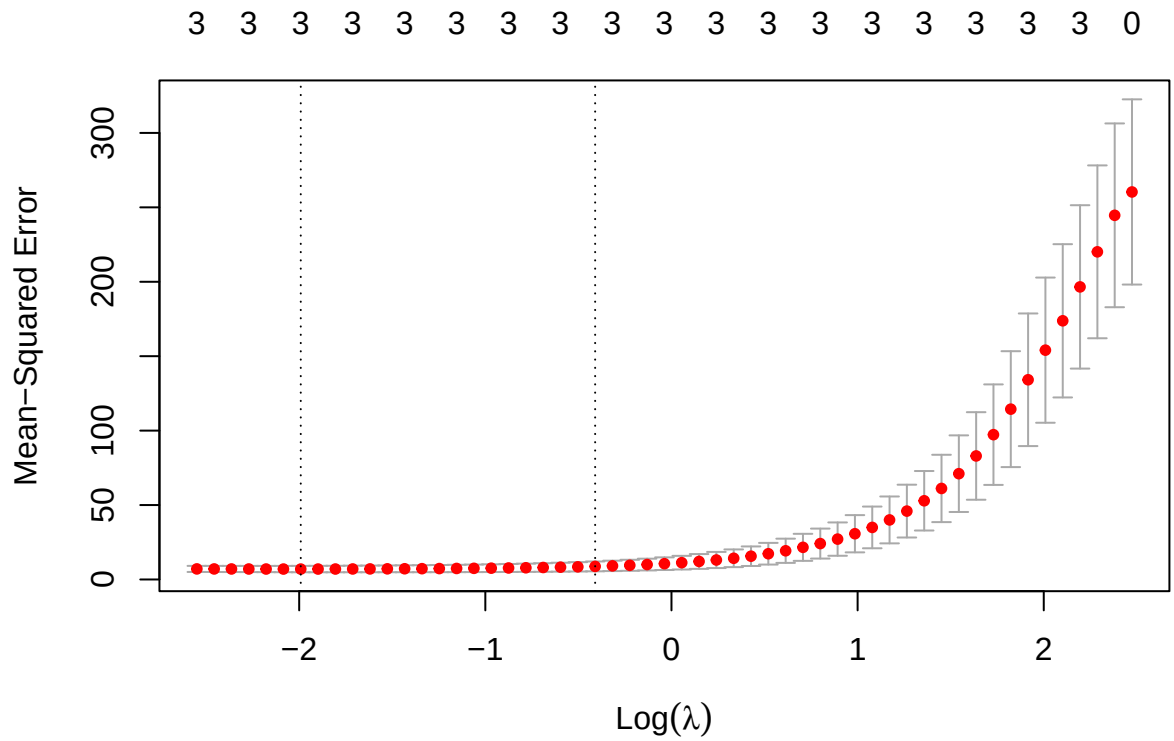
The best λ for the ridge regression model is 1.1871

c-)

```
LassoMod <- glmnet(x, y, alpha=1, nlambda=100,lambda.min.ratio=0.0001)
CvLassoMod <- cv.glmnet(x, y, alpha=1, nlambda=100,lambda.min.ratio=0.0001)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
plot(CvLassoMod)
```




```
best.lambda.lasso <- CvLassoMod$lambda.min
pred.lasso <- coef(CvLassoMod, s = "lambda.min")
pred.lasso
```

```
## 5 x 1 sparse Matrix of class "dgCMatrix"
##                      s1
## (Intercept) 71.9705570
## X1           1.4323974
## X2           0.4110560
## X3           .
## X4          -0.2343099
```

The best λ for the lasso regression model is 0.1365. This model gets rid of X_3.

d-)

```
EnetMod <- glmnet(x, y, alpha=0.5, nlambda=100,lambda.min.ratio=0.0001)
CvElasticnetMod <- cv.glmnet(x, y,alpha=0.5,nlambda=100,lambda.min.ratio=0.0001)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
best.lambda.enet <- CvElasticnetMod$lambda.min
pred.enet <- coef(CvElasticnetMod, s = "lambda.min")
pred.enet
```

```
## 5 x 1 sparse Matrix of class "dgCMatrix"
##                       s1
## (Intercept) 79.93061645
## X1           1.33931058
## X2           0.32927293
## X3          -0.08361242
## X4          -0.31241870
```

The best λ for the elastic net model is 0.273

e-)

```
yhat.ridge <- predict(RidgeMod, s = best.lambda.ridge, newx = x)
yhat.lasso <- predict(LassoMod, s = best.lambda.lasso, newx = x)
yhat.enet <- predict(EnetMod , s = best.lambda.enet, newx = x)
sst <- sum((y - mean(y))^2)
sse.ols<-sum(f1$residuals^2)
sse.ridge <- sum((y - yhat.ridge)^2)
sse.lasso <- sum((y - yhat.lasso)^2)
sse.enet <- sum((y - yhat.enet)^2)
tbl.sse <- cbind(sse.ols,sse.ridge,sse.lasso,sse.enet)
kable(tbl.sse, digits=4, align="c", caption="SSE Comparison", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 1: SSE Comparison

sse.ols	sse.ridge	sse.lasso	sse.enet
47.8636	56.1655	48.3656	49.0541

```
# R squared
rsq.ols <- 1 - sse.ols / sst
rsq.ridge <- 1 - sse.ridge / sst
rsq.lasso <- 1 - sse.lasso / sst
rsq.enet  <- 1 - sse.enet  / sst
tbl.r2 <- cbind(rsq.ols,rsq.ridge,rsq.lasso,rsq.enet)
kable(tbl.r2, digits=4, align="c", caption="R Squared Comparison", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 2: R Squared Comparison

rsq.ols	rsq.ridge	rsq.lasso	rsq.enet
0.9824	0.9793	0.9822	0.9819

SSE is the smallest for the ordinary least squares model (OLS), but the VIFs shown in part (a) indicate that there is multicollinearity issue. The optimal model of our selection would be the lasso regression model because it has the next smallest SSE, but the elastic net model comes pretty close. The lasso model is also preferred over the elastic net model since the lasso model uses one fewer predictor variable.

Problem 2

a-)

```
data(stackloss)
f2 <- lm(stack.loss ~ ., data=stackloss)
summary(f2)
```

```
##
## Call:
## lm(formula = stack.loss ~ ., data = stackloss)
##
```

```
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.2377 -1.7117 -0.4551  2.3614  5.6978
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -39.9197    11.8960  -3.356  0.00375 **
## Air.Flow      0.7156     0.1349   5.307 0.000058 ***
## Water.Temp    1.2953     0.3680   3.520  0.00263 **
## Acid.Conc.   -0.1521     0.1563  -0.973  0.34405
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.243 on 17 degrees of freedom
## Multiple R-squared:  0.9136, Adjusted R-squared:  0.8983
## F-statistic:  59.9 on 3 and 17 DF,  p-value: 0.000000003016
```

```
vif(f2)
```

```
##   Air.Flow Water.Temp Acid.Conc.
##   2.906484   2.572632   1.333587
```

R^2 is very big, and the model is significant. One predictor variable, Acid.Conc., looks not significant, and the VIFs show no multicollinearity problem.

b-)

```
rr.huber <- rlm(stack.loss ~ ., data = stackloss, psi = psi.huber)
summary(rr.huber)
```

```
##
## Call: rlm(formula = stack.loss ~ ., data = stackloss, psi = psi.huber)
## Residuals:
##      Min       1Q   Median       3Q      Max
## -8.91753 -1.73127  0.06187  1.54306  6.50163
##
## Coefficients:
##             Value    Std. Error t value
## (Intercept) -41.0265   9.8073    -4.1832
## Air.Flow      0.8294   0.1112     7.4597
## Water.Temp    0.9261   0.3034     3.0524
## Acid.Conc.   -0.1278   0.1289    -0.9922
##
## Residual standard error: 2.441 on 17 degrees of freedom
```

c-)

```
rr.bisquare <- rlm(stack.loss ~ ., data=stackloss, psi = psi.bisquare)
summary(rr.bisquare)
```

```
##
## Call: rlm(formula = stack.loss ~ ., data = stackloss, psi = psi.bisquare)
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10.4356  -1.7065  -0.2392   0.8797   6.9326
##
## Coefficients:
##             Value    Std. Error t value
```

```
## (Intercept) -42.2853   9.5316    -4.4363
## Air.Flow      0.9275   0.1081     8.5841
## Water.Temp    0.6507   0.2949     2.2068
## Acid.Conc.   -0.1123   0.1252    -0.8970
##
## Residual standard error: 2.282 on 17 degrees of freedom
```

d-)

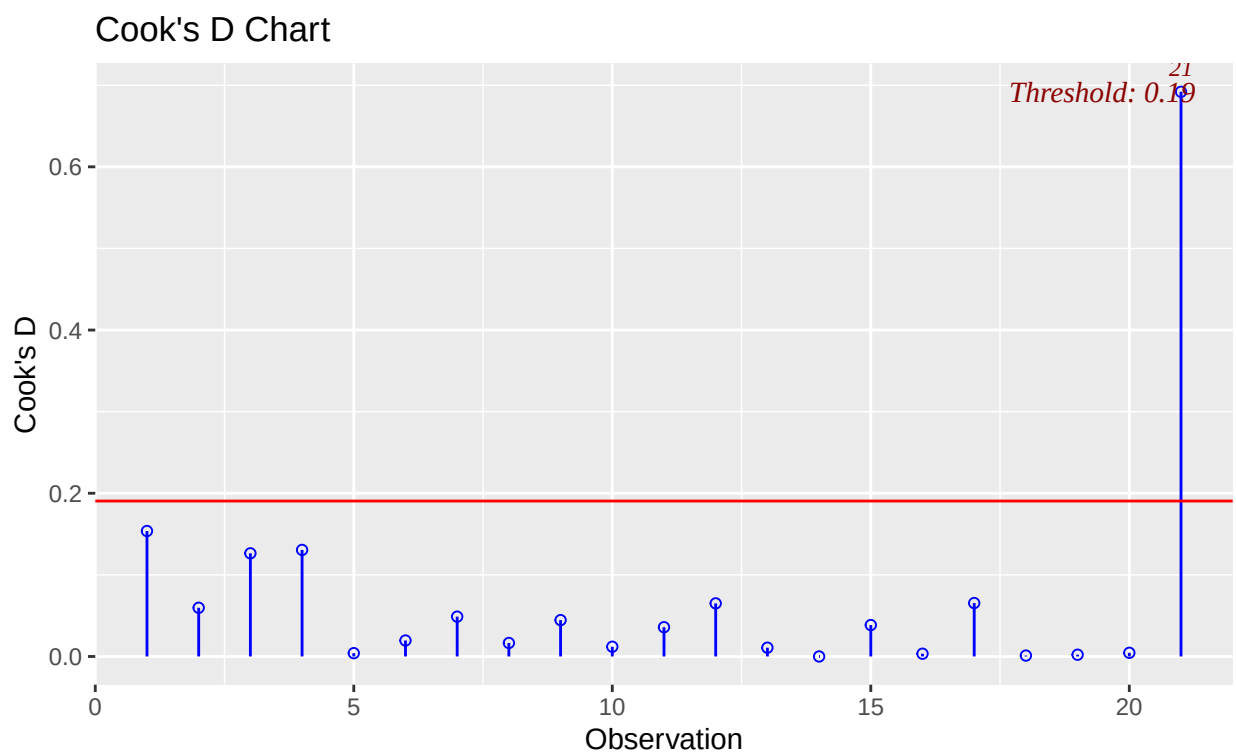
```
tbl.robust <- cbind(f2$coefficients, rr.huber$coefficients, rr.bisquare$coefficients)
colnames(tbl.robust) <- c("OLS", "Huber", "Bisquare")
kable(tbl.robust, digits=4, align="c", caption="OLS vs. Robust Comparison", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 3: OLS vs. Robust Comparison

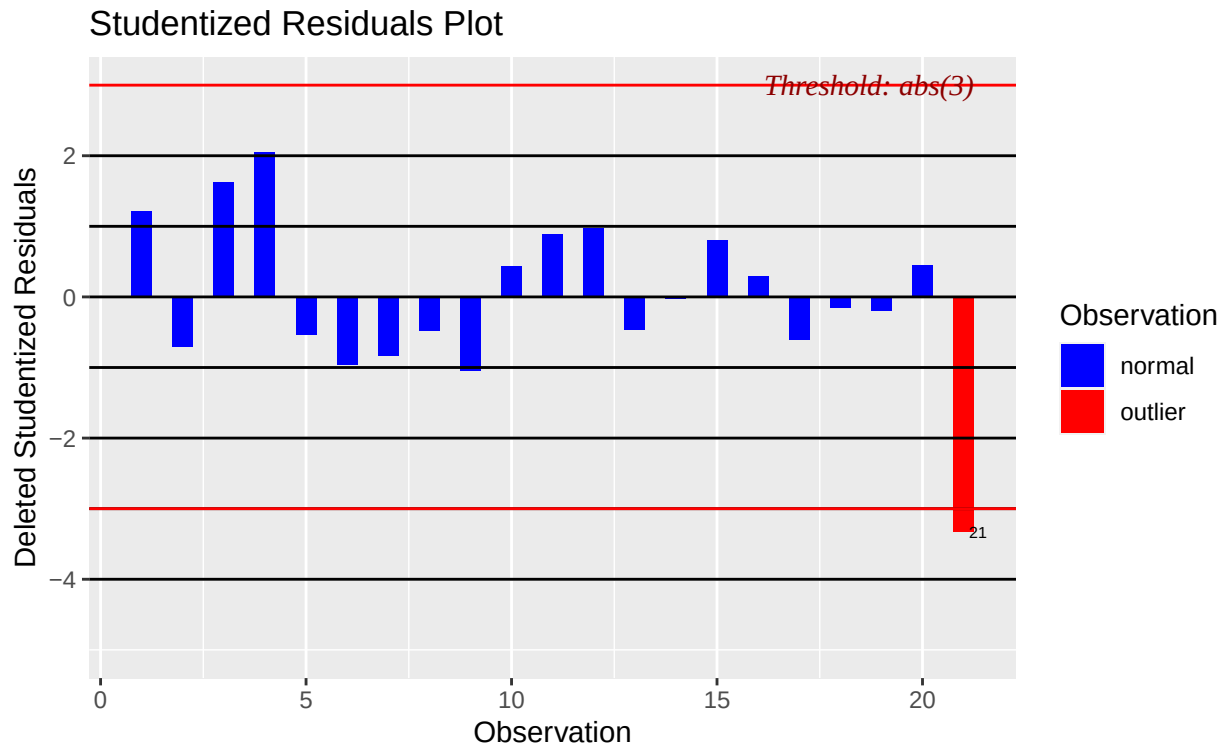
	OLS	Huber	Bisquare
(Intercept)	-39.9197	-41.0265	-42.2853
Air.Flow	0.7156	0.8294	0.9275
Water.Temp	1.2953	0.9261	0.6507
Acid.Conc.	-0.1521	-0.1278	-0.1123

Because the coefficients for OLS are quite different from those for robust regression models, especially for Water.Temp, we would use the robust regression models. It is an indication that there exist outliers or high leverage points.

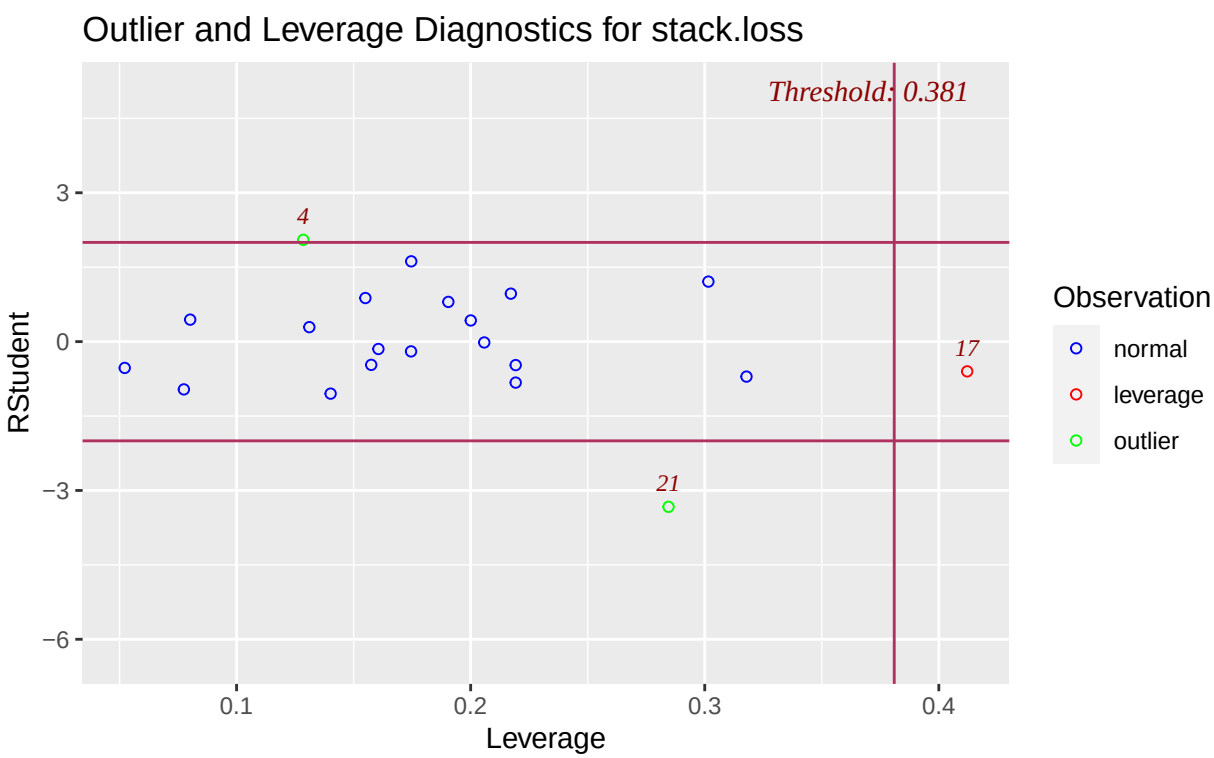
```
par(mfrow=c(2,2))
ols_plot_cooksd_chart(f2)
```



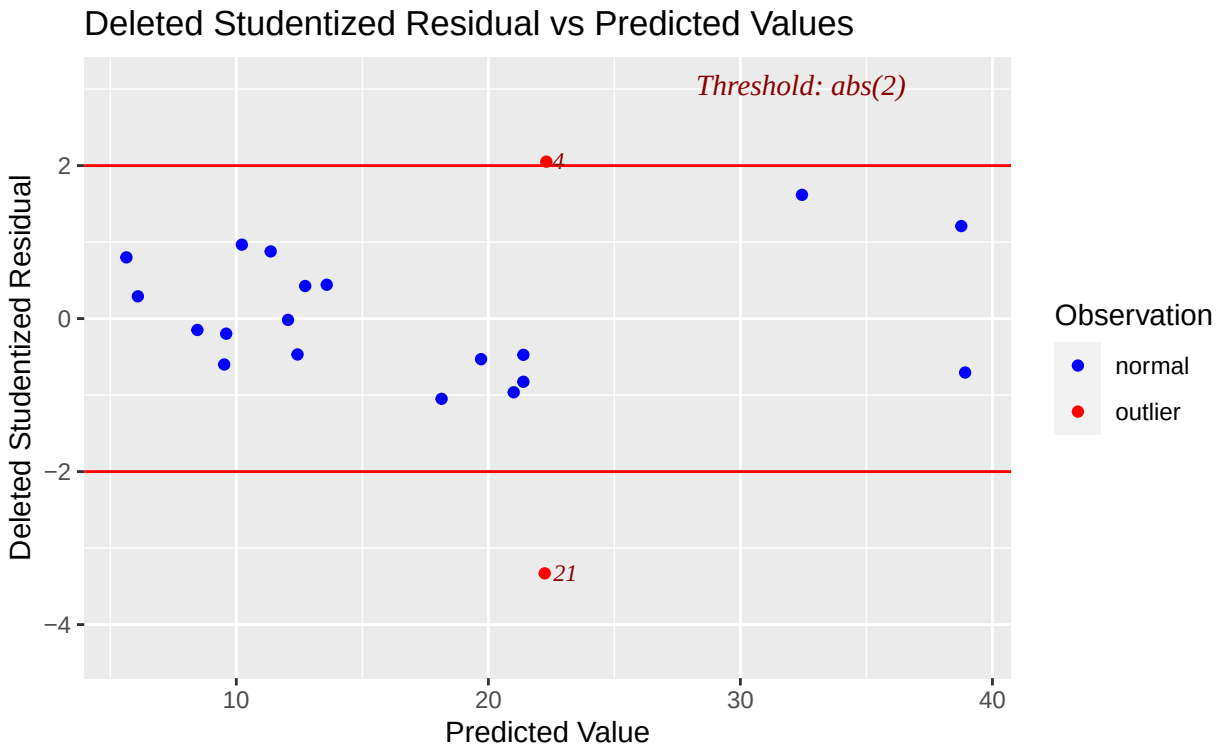
```
ols_plot_resid_stud(f2)
```




```
ols_plot_resid_lev(f2)
```



```r
ols_plot_resid_stud_fit(f2)
```



The four diagnostic plots show that the data point '21' could be an outlier. '4' is also identified as an outlier, but we will keep it in the model since it is on the border line. Also, the data set is quite small with only 21 points. We will also remove '17' for its high leverage.

```r
stackloss2 <- stackloss[-c(17,21),]
f2.1 <- lm(stack.loss ~ ., data=stackloss2)
summary(f2.1)
```

```
##
## Call:
## lm(formula = stack.loss ~ ., data = stackloss2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.0658 -2.0279 -0.0319  1.1301  6.2421
##
## Coefficients:
##             Estimate Std. Error t value   Pr(>|t|)
## (Intercept) -41.5706    12.4732  -3.333    0.00454 **
## Air.Flow      0.8859     0.1230   7.203 0.00000306 ***
## Water.Temp    0.8315     0.3392   2.452    0.02696 *
## Acid.Conc.   -0.1328     0.1585  -0.838    0.41539
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.647 on 15 degrees of freedom
## Multiple R-squared:  0.9465, Adjusted R-squared:  0.9358
## F-statistic: 88.48 on 3 and 15 DF,  p-value: 0.0000000009168
```

```
tbl.robust2 <- cbind(f2.1$coefficients, rr.huber$coefficients, rr.bisquare$coefficients)
colnames(tbl.robust2) <- c("OLS(New)", "Huber", "Bisquare")
kable(tbl.robust2, digits=4, align="c", caption="OLS vs. Robust Comparison", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 4: OLS vs. Robust Comparison

	OLS(New)	Huber	Bisquare
(Intercept)	-41.5706	-41.0265	-42.2853
Air.Flow	0.8859	0.8294	0.9275
Water.Temp	0.8315	0.9261	0.6507
Acid.Conc.	-0.1328	-0.1278	-0.1123

After removing an outlier and a high leverage point, we find that the new OLS has a lot similar coefficients to those of the robust regression models.

Problem 3

a-) Linear Regression

```
data(fat)
fat.data <- fat[,-c(1,3)]

set.seed(1023)
train.index <- createDataPartition(y = fat.data$siri, p = 0.7, list = FALSE)
train.set <- fat.data[train.index,]
test.set <- fat.data[-train.index,]

f3 <- lm(siri ~ ., data=train.set)
stepwise.result <- ols_step_both_p(f3, pent=0.1, prem=0.05)
stepwise.result
```

```
##
##                                Stepwise Selection Summary
## ---------------------------------------------------------------------------------------
##                       Added/                    Adj.
## Step    Variable     Removed     R-Square    R-Square      C(p)          AIC        RMSE
## ---------------------------------------------------------------------------------------
##    1      abdom      addition       0.717       0.716    1696.7770    1049.3015    4.5600
##    2       free      addition       0.865       0.863     721.0940     919.6851    3.1597
##    3      weight     addition       0.969       0.968      37.1010     661.5407    1.5259
##    4      adipos     addition       0.970       0.969      29.8350     655.3936    1.4957
##    5      biceps     addition       0.972       0.971      18.5270     644.9972    1.4487
##    6      thigh      addition       0.973       0.972      14.5200     641.1064    1.4291
##    7       age       addition       0.974       0.973      11.3600     637.8863    1.4124
##    8       neck      addition       0.974       0.973       9.0100     635.3628    1.3987
##    9      biceps     removal        0.974       0.973      10.8300     637.3412    1.4103
## ---------------------------------------------------------------------------------------
```

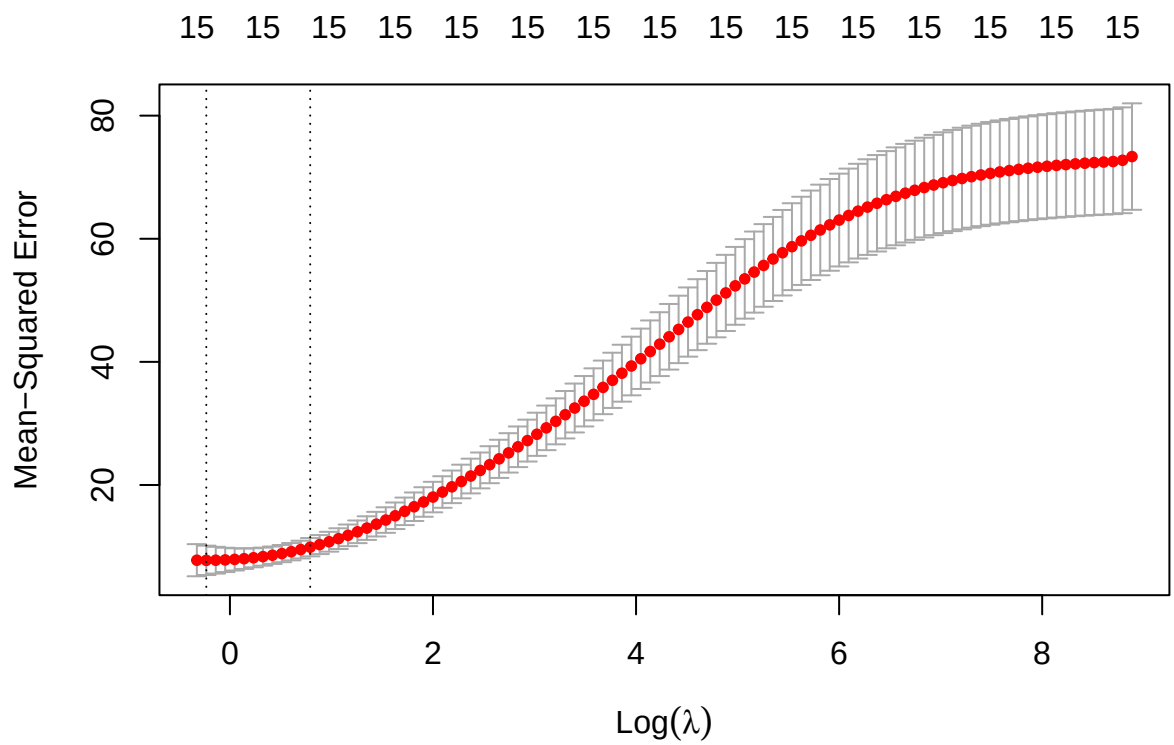
```
f3.ols <- lm(siri ~ abdom+free+weight+adipos+biceps+thigh+age+neck, data=train.set)
summary(f3.ols)
```

```
##
```

```
## Call:
## lm(formula = siri ~ abdom + free + weight + adipos + biceps +
##     thigh + age + neck, data = train.set)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.8364 -0.5812  0.1723  0.8395  7.3593
##
## Coefficients:
##             Estimate Std. Error t value  Pr(>|t|)
## (Intercept) -0.59899    3.26172  -0.184  0.854514
## abdom        0.13364    0.03845   3.475  0.000648 ***
## free        -0.53461    0.01415 -37.785   < 2e-16 ***
## weight       0.39703    0.01874  21.186   < 2e-16 ***
## adipos      -0.40450    0.08948  -4.520 0.0000116 ***
## biceps       0.13839    0.07081   1.954  0.052302 .
## thigh        0.17802    0.04963   3.587  0.000437 ***
## age          0.02341    0.01114   2.102  0.037056 *
## neck         0.19330    0.09268   2.086  0.038516 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.399 on 169 degrees of freedom
## Multiple R-squared:  0.9744, Adjusted R-squared:  0.9732
## F-statistic: 805.6 on 8 and 169 DF,  p-value: < 2.2e-16
```

c-) Ridge Regression

```r
x3 <- model.matrix(siri~., train.set)[,-c(1)]
y3 <- train.set$siri
RidgeMod3 <- glmnet(x3, y3, alpha=0, nlambda=100,lambda.min.ratio=0.0001)
CvRidgeMod3 <- cv.glmnet(x3, y3, alpha=0, nlambda=100,lambda.min.ratio=0.0001)
par(mfrow=c(1,1))
plot(CvRidgeMod3)
```

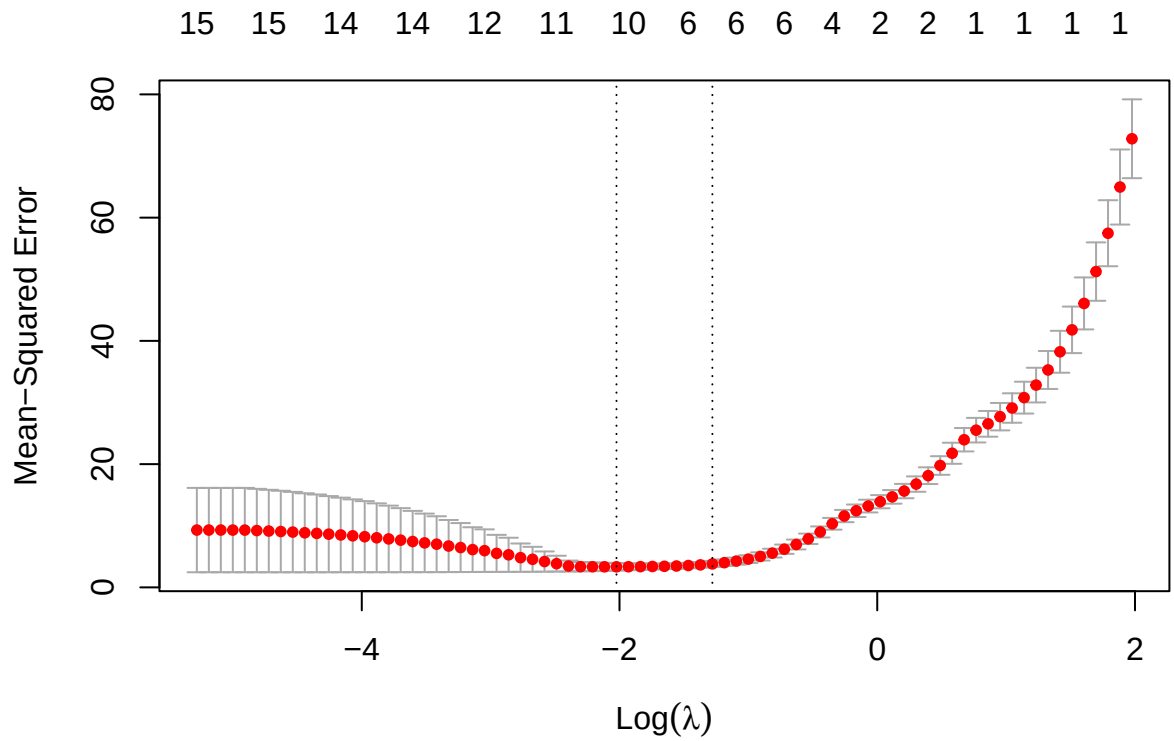



```
best.lambda.ridge3 <- CvRidgeMod3$lambda.min
pred.ridge3 <- coef(CvRidgeMod3, s = "lambda.min")
pred.ridge3
```

```
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                       s1
## (Intercept) -49.43929724
## age           0.04378042
## weight        0.08923571
## height        0.11821724
## adipos        0.17517009
## free         -0.33524201
## neck          0.24982841
## chest         0.09158334
## abdom         0.26394061
## hip           0.16463142
## thigh         0.21291848
## knee          0.28059377
## ankle         0.15682564
## biceps        0.14391669
## forearm       0.10070794
## wrist        -0.44465593
```

d-) Lasso

```
LassoMod3 <- glmnet(x3, y3, alpha=1, nlambda=100,lambda.min.ratio=0.0001)
CvLassoMod3 <- cv.glmnet(x3, y3, alpha=1, nlambda=100,lambda.min.ratio=0.0001)
plot(CvLassoMod3)
```

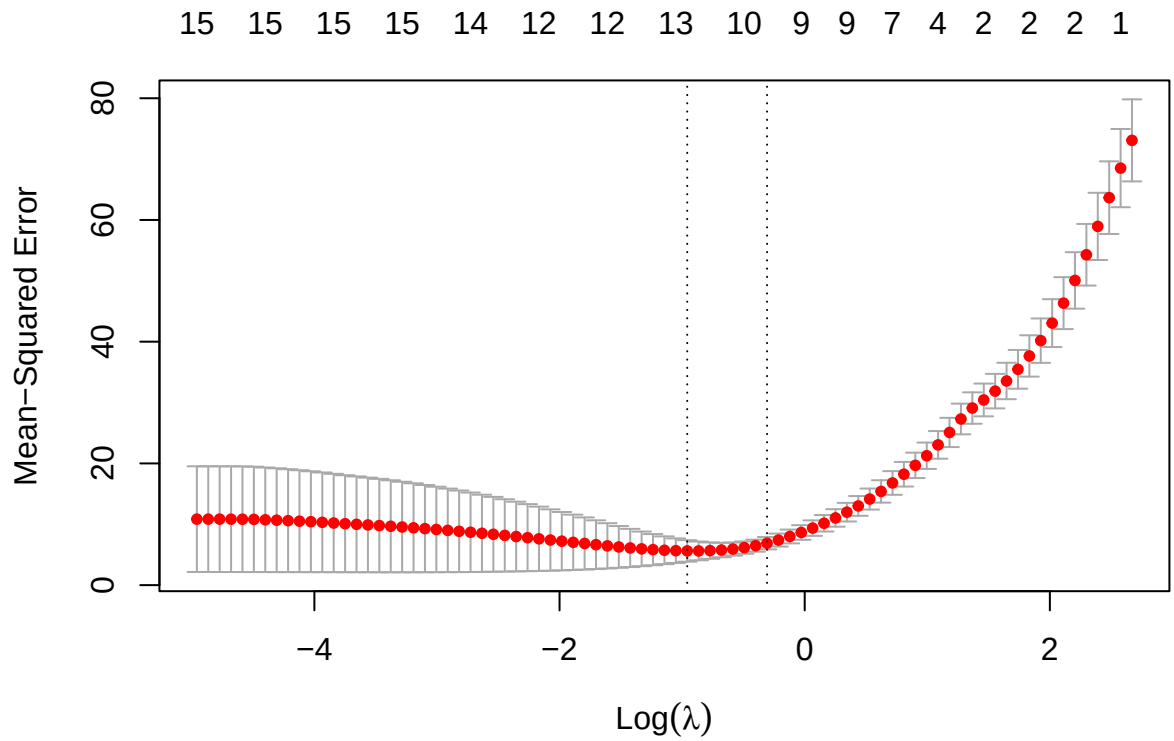



```
best.lambda.lasso3 <- CvLassoMod3$lambda.min
pred.lasso3 <- coef(CvLassoMod3, s = "lambda.min")
pred.lasso3
```

```
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                        s1
## (Intercept) -6.424104731
## age          0.004220293
## weight       0.308460507
## height       0.031378751
## adipos       .
## free        -0.462976897
## neck         0.061793183
## chest        .
## abdom        0.192576379
## hip          .
## thigh        0.088319904
## knee         0.142810083
## ankle        0.020084485
## biceps       0.096792626
## forearm      0.000314814
## wrist        .
```

e-) Elastic Net

```
EnetMod3 <- glmnet(x3, y3, alpha=0.5, nlambda=100,lambda.min.ratio=0.0001)
CvElasticnetMod3 <- cv.glmnet(x3, y3,alpha=0.5,nlambda=100,lambda.min.ratio=0.0001)
plot(CvElasticnetMod3)
```




```
best.lambda.enet3 <- CvElasticnetMod3$lambda.min
pred.enet3 <- coef(CvElasticnetMod3, s = "lambda.min")
pred.enet3
```

```
## 16 x 1 sparse Matrix of class "dgCMatrix"
##                          s1
## (Intercept) -23.5338050323
## age           0.0001450146
## weight        0.1719963102
## height        0.0131406703
## adipos        .
## free         -0.3640714545
## neck          0.1159516194
## chest         0.0060174894
## abdom         0.3482023175
## hip           0.0505006653
## thigh         0.1152427062
## knee          0.2136943717
## ankle         0.0379457604
## biceps        0.1418193126
## forearm       0.0167839748
## wrist         .
```

f-) Robust Regression

```
rr.huber3 <- rlm(siri ~ ., data = train.set, psi = psi.huber)
summary(rr.huber3)
```

```
##
## Call: rlm(formula = siri ~ ., data = train.set, psi = psi.huber)
## Residuals:
##     Min      1Q  Median      3Q     Max
```

```
## -5.6692 -0.5584  0.1067  0.6925  8.1042
##
## Coefficients:
##             Value    Std. Error t value
## (Intercept)  -1.5202   4.9770    -0.3054
## age           0.0221   0.0097     2.2669
## weight        0.4225   0.0175    24.1663
## height        0.0383   0.0276     1.3907
## adipos       -0.3372   0.0893    -3.7738
## free         -0.5645   0.0111   -50.7489
## neck          0.1648   0.0788     2.0898
## chest         0.0120   0.0334     0.3578
## abdom         0.0788   0.0319     2.4693
## hip           0.0229   0.0418     0.5488
## thigh         0.1817   0.0445     4.0817
## knee         -0.0148   0.0723    -0.2047
## ankle         0.1033   0.0571     1.8107
## biceps        0.0362   0.0562     0.6442
## forearm       0.0694   0.0616     1.1260
## wrist        -0.0914   0.1567    -0.5835
##
## Residual standard error: 0.9743 on 162 degrees of freedom
```

```
rr.bisquare3 <- rlm(siri ~ ., data=train.set, psi = psi.bisquare)
```

```
## Warning in rlm.default(x, y, weights, method = method, wt.method = wt.method, :
## 'rlm' failed to converge in 20 steps
```

```
summary(rr.bisquare3)
```

```
##
## Call: rlm(formula = siri ~ ., data = train.set, psi = psi.bisquare)
## Residuals:
##     Min      1Q  Median      3Q     Max
## -6.4025 -0.6409  0.1040  0.5667  8.7422
##
## Coefficients:
##             Value    Std. Error t value
## (Intercept)   3.0994   4.7229     0.6562
## age           0.0189   0.0092     2.0400
## weight        0.4498   0.0166    27.1103
## height        0.0419   0.0262     1.6021
## adipos       -0.2278   0.0848    -2.6869
## free         -0.5795   0.0106   -54.9009
## neck          0.1355   0.0748     1.8108
## chest        -0.0065   0.0317    -0.2046
## abdom         0.0503   0.0303     1.6599
## hip           0.0305   0.0396     0.7705
## thigh         0.1698   0.0423     4.0198
## knee         -0.1287   0.0687    -1.8743
## ankle         0.1062   0.0541     1.9614
## biceps       -0.0296   0.0534    -0.5538
## forearm       0.0663   0.0585     1.1341
## wrist         0.0159   0.1487     0.1072
##
```

```
## Residual standard error: 0.8748 on 162 degrees of freedom
```

g-)

```
xt <- model.matrix(siri~., test.set)[,-c(1)]
yhat.ridge3 <- predict(RidgeMod3, s = best.lambda.ridge3, newx = xt)
yhat.lasso3 <- predict(LassoMod3, s = best.lambda.lasso3, newx = xt)
yhat.enet3 <- predict(EnetMod3, s = best.lambda.enet3, newx = xt)
yhat.huber <- predict(rr.huber3, test.set)
yhat.bisquare <- predict(rr.bisquare3, test.set)

y4 <- test.set$siri
sse.ols3 <- sum(f3.ols$residuals^2)
sse.ridge3 <- sum((y4 - yhat.ridge3)^2)
sse.lasso3 <- sum((y4 - yhat.lasso3)^2)
sse.enet3 <- sum((y4 - yhat.enet3)^2)
sse.huber <- sum((y4 - yhat.huber)^2)
sse.bisquare <- sum((y4 - yhat.bisquare)^2)

tbl.sse3 <- cbind(sse.ols3,sse.ridge3,sse.lasso3,sse.enet3,sse.huber,sse.bisquare)
colnames(tbl.sse3) <- c("OLS","Ridge","Lasso","ENet","Huber","Bisquare")
kable(tbl.sse3, digits=3, align="c", caption="SSE Comparison", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 5: SSE Comparison

OLS	Ridge	Lasso	ENet	Huber	Bisquare
330.63	587.242	375.169	497.239	328.703	373.264

The Huber model has the smallest SSE and would be the best candidate for the final selection although the Lasso and Bisquare models' performances are also not bad. The OLS also has the similar SSE as the Huber, but VIFs shown below indicate multicollinearity issue.

```
vif(f3.ols)
```

```
##     abdom      free    weight    adipos    biceps     thigh       age      neck
## 15.635610  5.487222 25.060511  9.212945  4.258302  6.062638  1.922709  4.075904
```

Problem 4

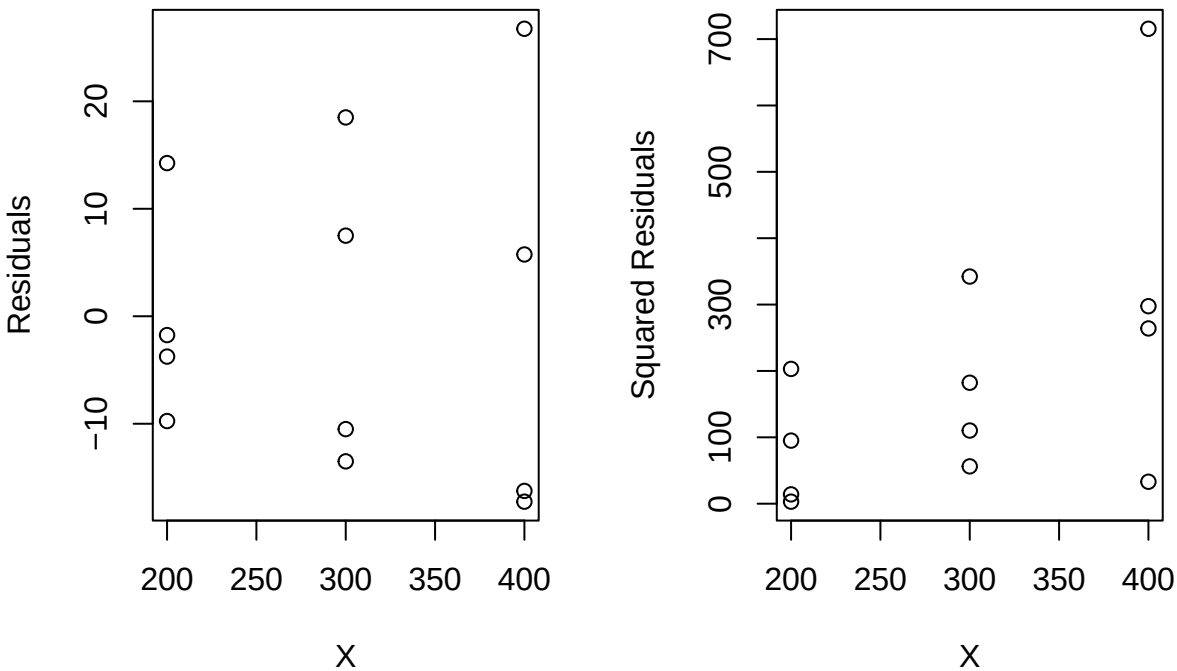
a-)

```
MSdata <- getDataSet("Machine Speed.csv")
f4 <- lm(Y ~ X, data=MSdata)
summary(f4)
```

```
##
## Call:
## lm(formula = Y ~ X, data = MSdata)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -17.250 -11.250  -2.750   9.188  26.750
##
## Coefficients:
```

```
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.75000   16.73052  -0.344  0.73820
## X            0.18750    0.05381   3.484  0.00588 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.22 on 10 degrees of freedom
## Multiple R-squared:  0.5484, Adjusted R-squared:  0.5032
## F-statistic: 12.14 on 1 and 10 DF,  p-value: 0.005878
```

```
ei <- f4$residuals
par(mfrow=c(1,2))
plot(MSdata$X, ei, xlab="X", ylab="Residuals")
plot(MSdata$X, ei^2, xlab="X", ylab="Squared Residuals")
```



The plots above show a megaphone shape, indicating that the error variances increase as X gets larger. But, since the size of the data set is quite small, we would also want to check the significance with some formal test.

b-)

```
bptest(f4)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  f4
## BP = 3.3723, df = 1, p-value = 0.0663
```

H_0: The error variances are constant.
H_a: The error variances are not constant.

The p-value of Breusch-Pagan test is smaller than α=0.1, so we conclude H_a that the error variances are not constant. It is consistent with what we observed with residual plots in part (a).

c-)

```r
abs.ei <- abs(ei)
g1 <- lm(abs.ei ~ MSdata$X)
s <- g1$fitted.values
w1 <- 1/(s^2)
#use the weights to fit weighted regression model
g2 <- lm(Y ~ X, weights= w1, data=MSdata)
summary(g2)
```

```
##
## Call:
## lm(formula = Y ~ X, data = MSdata, weights = w1)
##
## Weighted Residuals:
##     Min      1Q  Median      3Q     Max
## -1.2685 -1.0009 -0.3429  0.8435  1.9051
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -6.26646   13.35177  -0.469  0.64890
## X            0.18930    0.05133   3.688  0.00419 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.221 on 10 degrees of freedom
## Multiple R-squared:  0.5763, Adjusted R-squared:  0.5339
## F-statistic:  13.6 on 1 and 10 DF,  p-value: 0.004192
```

```r
tbl.4c <- data.frame(cbind(f4$coefficients, g2$coefficients))
colnames(tbl.4c) <- c("Ordinary","Weighted")
kable(tbl.4c, digits=4, align="c", caption="Ordinary vs. Weighted Least Squares", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 6: Ordinary vs. Weighted Least Squares

	Ordinary	Weighted
(Intercept)	-5.7500	-6.2665
X	0.1875	0.1893

For β_1, it is almost identical for both models. For β_0, it is changed a little bit but does not appear to be significantly different.

e-)

```r
tbl.4e <- data.frame(cbind(summary(f4)$coefficients[,2], summary(g2)$coefficients[,2]))
colnames(tbl.4e) <- c("Ordinary","Weighted")
kable(tbl.4e, digits=4, align="c", caption="Standard Deviations of Estimated Coefficients", booktabs=TRU
  kable_styling(latex_options="HOLD_position")
```

Table 7: Standard Deviations of Estimated Coefficients

	Ordinary	Weighted
(Intercept)	16.7305	13.3518
X	0.0538	0.0513

For β_1, SD is almost identical for both models. For β_0, SD is reduced slightly.

f-)

```
ei2 <- g2$residuals
abs.ei2 <- abs(ei2)
g3 <- lm(abs.ei2 ~ MSdata$X)
s2 <- g3$fitted.values
w2 <- 1/(s2^2)
#use the weights to fit weighted regression model
g4 <- lm(Y ~ X, weights= w2, data=MSdata)
summary(g4)
```

```
##
## Call:
## lm(formula = Y ~ X, data = MSdata, weights = w2)
##
## Weighted Residuals:
##     Min      1Q  Median      3Q     Max
## -1.2794 -1.0002 -0.3457  0.8453  1.9219
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -6.27128   13.33796  -0.470  0.64831
## X            0.18931    0.05139   3.684  0.00422 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.226 on 10 degrees of freedom
## Multiple R-squared:  0.5757, Adjusted R-squared:  0.5333
## F-statistic: 13.57 on 1 and 10 DF,  p-value: 0.00422
```

```
tbl.4f <- data.frame(cbind(f4$coefficients, g2$coefficients, g4$coefficients))
colnames(tbl.4f) <- c("Ordinary","Weighted1","Weighted2")
kable(tbl.4f, digits=4, align="c", caption="Ordinary vs. Weighted Least Squares", booktabs=TRUE) %>%
  kable_styling(latex_options="HOLD_position")
```

Table 8: Ordinary vs. Weighted Least Squares

	Ordinary	Weighted1	Weighted2
(Intercept)	-5.7500	-6.2665	-6.2713
X	0.1875	0.1893	0.1893

There is no substantial change in the estimated regression coefficients from the first weighted least squares model. In fact, it is almost the same. If there were a big change, we should iterate the previous steps until

the estimates of coefficients converge. But, here there is no need for iteration since it seems to be already converged.